

Proposta de Domínio — PlantOS

Disciplina: Lab. de Desenvolvimento de Aplicações Móveis e Distribuídas — PUC Minas

Curso: Engenharia de Software — 5º Período

Semestre: 1º Semestre 2026

Sprint: 1 — Arquitetura e Backend REST

1. Domínio Escolhido

PlantOS é um sistema distribuído de **ordens de serviço (OS)** para **plantas industriais**. O sistema permite que operadores de chão de fábrica registrem falhas, anomalias ou necessidades de manutenção em equipamentos e instalações, e que técnicos de manutenção recebam, aceitem e executem essas ordens de forma organizada e assíncrona.

O nome “PlantOS” é uma combinação de *Plant* (planta industrial) e *OS* (ordem de serviço), refletindo o propósito central da aplicação.

2. Justificativa

Em ambientes industriais, a comunicação entre operadores e equipes de manutenção é frequentemente feita de forma verbal, por rádio ou em quadros físicos — métodos sujeitos a perda de informação, atrasos e falta de rastreabilidade. Falhas não documentadas podem resultar em paradas de produção prolongadas, aumento de custos operacionais e riscos à segurança dos trabalhadores.

O PlantOS resolve esse problema ao digitalizar o fluxo de abertura, acompanhamento e conclusão de ordens de serviço, proporcionando:

- **Rastreabilidade completa:** cada OS possui histórico de criação, aceite e conclusão com laudo técnico.
- **Comunicação assíncrona em tempo real:** operadores e técnicos são notificados automaticamente sobre mudanças de estado via mensageria (RabbitMQ), eliminando a necessidade de consultas manuais.
- **Registro de materiais:** técnicos documentam os materiais utilizados em cada manutenção, facilitando o controle de estoque e a análise de custos.
- **Mobilidade:** aplicativos móveis permitem que tanto operadores quanto técnicos atuem diretamente do chão de fábrica, sem depender de terminais fixos.

O domínio satisfaz a estrutura **cliente/prestador** exigida pelo projeto: o operador atua como cliente (solicitante do serviço) e o técnico como prestador (executor da manutenção).

3. Perfis de Usuário

3.1 Operador (Cliente)

O operador é o profissional que atua diretamente na operação da planta industrial (produção, monitoramento de processos, controle de qualidade). Ao identificar uma falha

ou necessidade de manutenção, ele utiliza o aplicativo móvel para abrir uma ordem de serviço.

Funcionalidade	Descrição
Abrir OS	Registra uma nova ordem de serviço com título, descrição da falha, setor/equipamento afetado e nível de prioridade (baixa, média, alta, crítica)
Listar OS	Visualiza todas as ordens de serviço que abriu, com filtros por status
Acompanhar status	Monitora em tempo real o progresso da OS (aberta → aceita → em andamento → concluída)
Receber notificações	É notificado automaticamente quando o técnico aceita, recusa ou conclui a OS
Visualizar laudo	Consulta o laudo técnico e os materiais registrados após a conclusão

3.2 Técnico de Manutenção (Prestador)

O técnico é o profissional responsável por executar as manutenções corretivas e preventivas na planta. Ele recebe as ordens de serviço abertas pelos operadores e decide quais aceitar com base em sua disponibilidade e especialidade.

Funcionalidade	Descrição
Receber OS	É notificado em tempo real sobre novas ordens de serviço abertas
Aceitar/Recusar OS	Avalia a OS e decide se aceita ou recusa a execução
Registrar materiais	Documenta os materiais e peças utilizados durante a manutenção
Concluir com laudo	Finaliza a OS registrando um laudo técnico descrevendo o serviço realizado
Histórico de OS	Consulta as ordens de serviço que executou anteriormente

4. Principais Funcionalidades do Sistema

4.1 Gestão de Ordens de Serviço

- **Criação:** operador informa título, descrição, setor, equipamento e prioridade.
- **Ciclo de vida:** cada OS transita pelos estados **aberta** → **aceita** → **em_andamento** → **concluída** (ou **recusada**).
- **Consulta por ID:** qualquer usuário pode consultar os detalhes de uma OS específica.
- **Listagem com filtros:** listagem de todas as OS com possibilidade de filtrar por status.

4.2 Comunicação Assíncrona (Eventos)

O sistema é orientado a eventos. Toda mudança de estado de uma OS gera um evento publicado no RabbitMQ:

Evento	Produzido quando	Consumidor
os.criada	Operador abre nova OS	App do técnico
os.aceita	Técnico aceita a OS	App do operador
os.recusada	Técnico recusa a OS	App do operador
os.em_andamento	Técnico inicia a execução	App do operador
os.concluida	Técnico conclui com laudo	App do operador

4.3 Registro de Materiais e Laudo

Ao concluir uma OS, o técnico registra:

- Lista de materiais utilizados (nome, quantidade)
- Laudo técnico descritivo do serviço executado

Esses dados ficam vinculados à OS e disponíveis para consulta pelo operador.

5. Modelo de Dados (Visão Geral)

Ordem de Serviço (ordem_servico)

Campo	Tipo	Descrição
id	INTEGER (PK)	Identificador único
titulo	TEXT	Título resumido da falha
descricao	TEXT	Descrição detalhada
setor	TEXT	Setor ou área da planta
equipamento	TEXT	Equipamento afetado
prioridade	TEXT	baixa, média, alta, crítica
status	TEXT	aberta, aceita, em_andamento, concluída, recusada
operador_id	TEXT	Identificador do operador
tecnico_id	TEXT	Identificador do técnico (após aceite)
laudo	TEXT	Laudo técnico (após conclusão)
criado_em	DATETIME	Data/hora de criação
atualizado_em	DATETIME	Data/hora da última atualização

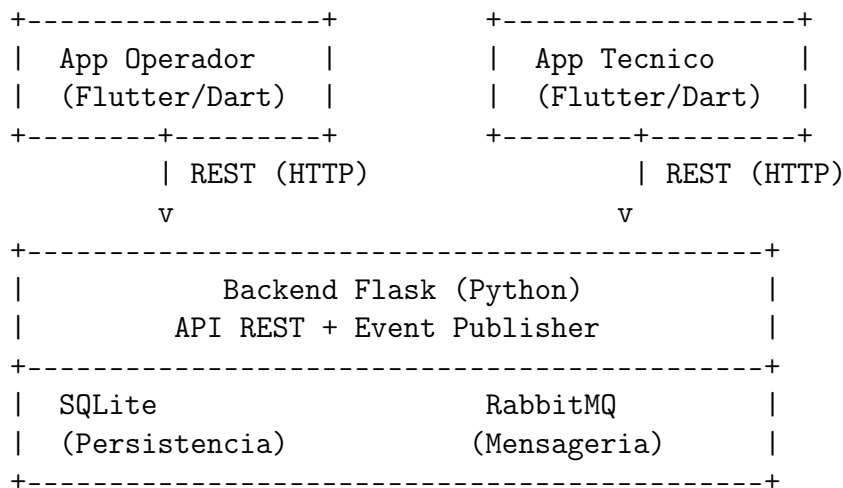
Material Utilizado (material)

Campo	Tipo	Descrição
id	INTEGER (PK)	Identificador único
ordem_servico_id	INTEGER (FK)	Referência à OS
nome	TEXT	Nome do material
quantidade	INTEGER	Quantidade utilizada

Método	Rota	Descrição
POST	/os	Criar nova ordem de serviço
GET	/os	Listar todas as OS (com filtros opcionais por status)
GET	/os/<id>	Consultar OS por ID
PATCH	/os/<id>/aceitar	Técnico aceita a OS
PATCH	/os/<id>/recusar	Técnico recusa a OS
PATCH	/os/<id>/iniciar	Técnico inicia a execução da OS
PATCH	/os/<id>/concluir	Técnico conclui a OS com laudo
POST	/os/<id>/materiais	Registrar materiais utilizados
GET	/os/<id>/materiais	Listar materiais de uma OS

6. Endpoints REST Planejados

7. Arquitetura do Sistema



Protocolos de comunicação:

- **Apps ↔ Backend:** HTTP/REST (JSON)
- **Backend ↔ Apps (eventos):** AMQP via RabbitMQ
- **Backend ↔ Banco:** SQLite (acesso local)

8. Tecnologias Utilizadas

Camada	Tecnologia	Justificativa
Apps móveis	Flutter 3.10+ / Dart 3.x	Framework multiplataforma com hot reload e UI declarativa
Backend	Flask (Python 3.11+)	Microframework leve, ideal para APIs REST
Banco de dados	SQLite	Embutido, sem necessidade de servidor separado
Mensageria (MOM)	RabbitMQ 3.12	Broker robusto com suporte a AMQP e painel de administração
Infraestrutura	Docker Compose	Containerização do RabbitMQ para ambiente reproduzível